

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

-----□□□□-----



ASSIGNMENT 03

Giảng viên hướng dẫn: Trần Đình Quế

Sinh viên thực hiện: Bùi Hồng Phú

Mã sinh viên: B23DCCN651

Hà Nội, 2026

Phần I. LÝ THUYẾT

1.1. Tổng quan về trí tuệ nhân tạo và hệ thống thông minh

- Trí tuệ được tiếp cận theo góc nhìn chức năng thay vì cố đưa ra một định nghĩa duy nhất. Một hệ thống thông minh có thể được nhìn qua chuỗi năng lực:

**quan sát → biểu diễn → học → suy luận → quyết định → hành động
→ nhận phản hồi**

Như vậy, trí tuệ nhân tạo không chỉ nằm ở việc xây dựng một thuật toán dự đoán, mà còn liên quan đến cách hệ thống biểu diễn thông tin, học từ dữ liệu, đưa ra quyết định và được triển khai trong thực tế.

Từ góc nhìn này, xây dựng một tiến trình từ biểu diễn dữ liệu đến khả năng học và hành động. Lộ trình được mô tả qua các giai đoạn: feature vectors và Machine Learning, representation learning và Deep Learning, tensors và graphs, embeddings và context, và cuối cùng là các hệ thống tích hợp.

Một điểm quan trọng được nhấn mạnh là quá trình phát triển AI cũng có thể được nhìn như quá trình phát triển của representation. Ban đầu hệ thống sử dụng symbols và rules; sau đó chuyển sang feature vectors trong Statistical Machine Learning; Deep Learning tập trung vào learned representations trên vectors, tensors và sequences; các hệ thống hiện đại tiếp tục mở rộng sang embeddings và multimodal representations.

Ví dụ: với hệ thống luật đơn giản, kiến thức có thể được biểu diễn trực tiếp:

IF fever AND cough → THEN possible_infection

Điểm mạnh của cách tiếp cận này là kiến thức và quá trình suy luận có thể quan sát được rõ ràng. Tuy nhiên, hệ thống dễ bị hạn chế khi tri thức không đầy đủ. Điều này dẫn đến một thay đổi quan trọng: thay vì chỉ lập trình tri thức hoặc đặc trưng, hệ thống ngày càng chuyển sang học cách biểu diễn từ dữ liệu.

1.2. Machine Learning và vai trò của biểu diễn dữ liệu

- Trong Machine Learning truyền thống, dữ liệu thường được chuyển thành một vector đặc trưng trước khi đưa vào thuật toán học. Với dữ liệu có cấu trúc, quy trình có thể được mô tả:

Raw Data → Prepared Features → ML Algorithm → Prediction

Cách tiếp cận này có ưu điểm là biểu diễn tương đối rõ ràng, dễ kiểm tra

và các thuật toán có thể được so sánh trong cùng một không gian đặc trưng. Đây cũng là lý do các bài toán dữ liệu có cấu trúc được sử dụng làm nền tảng ban đầu trong môn học.

Ví dụ: trong bài toán dự đoán bệnh tiểu đường, các đặc trưng như glucose, blood pressure, BMI hay age có ý nghĩa trực tiếp đối với con người. Những thuật toán khác nhau có thể nhận cùng một feature vector nhưng sử dụng các cơ chế học khác nhau, chẳng hạn Logistic Regression, KNN, SVM, Decision Tree và Random Forest.

Tuy nhiên, cách tiếp cận này có một hạn chế: con người phải thiết kế đặc trưng. Với dữ liệu phức tạp như hình ảnh, rất khó có thể thủ công xác định toàn bộ các đặc trưng có ý nghĩa. Từ đó xuất hiện nhu cầu về Representation Learning.

1.3. Representation Learning và sự chuyển dịch sang Deep Learning

- Sự khác biệt cốt lõi giữa Machine Learning truyền thống và Deep Learning nằm ở cách biểu diễn được tạo ra.

Machine Learning truyền thống: Human-designed Features → Prediction Model

Deep Learning: Data → Learned Representation → Prediction

Trong Deep Learning, mô hình không chỉ học mối quan hệ giữa đặc trưng và đầu ra mà còn học cách biến đổi biểu diễn ban đầu thành những biểu diễn trung gian hữu ích cho nhiệm vụ.

$$x \rightarrow h_{\theta}(x) \rightarrow \hat{y}$$

trong đó:

- + x là đầu vào
- + h là learned representation
- + $\hat{y}$ là kết quả dự đoán.

Biểu diễn h không được lập trình viên định nghĩa trước mà được quyết định bởi các tham số học được từ dữ liệu. Đây chính là ý tưởng nền tảng của Deep Learning: mô hình đồng thời học representation và prediction function.

1.4. Neural Network là gì?

- Một Neural Network có thể được hiểu là một hàm toán học có tham số, được xây dựng bằng cách nối tiếp nhiều phép biến đổi. Một mạng đơn giản có thể được viết:

$$f_{\theta}(x) = W_2 \varphi(W_1 x + b_1) + b_2$$

trong đó:

- + W là trọng số
- + b là bias
- + φ là activation function
- + θ là tập các tham số cần học.

=> Do đó, Neural Network không phải là một cơ chế bí ẩn; về bản chất, đó là một hàm toán học mà các tham số được học từ dữ liệu.

1.5. Neuron và phép biến đổi tuyến tính

Một neuron đơn lẻ nhận một vector đầu vào $x = [x_1, x_2, \dots, x_d]^T$ và trước tiên thực hiện phép biến đổi tuyến tính:

$$z = w^T x + b$$

Sau đó áp dụng hàm kích hoạt:

$$h = \varphi(z)$$

Điểm quan trọng là neuron kết hợp hai thành phần: phép biến đổi tuyến tính và tính phi tuyến.

Linear Transformation + Nonlinearity

Khi một layer có nhiều neuron, phép tính có thể viết dưới dạng ma trận:

$$z = Wx + b$$

Với:

$$x \in \mathbb{R}^d, W \in \mathbb{R}^{k \times d} \text{ và } b \in \mathbb{R}^k$$

thì:

$$z \in \mathbb{R}^k$$

=> Như vậy layer thực hiện phép biến đổi từ $\mathbb{R}^d$ sang $\mathbb{R}^k$ và đồng thời tạo ra một biểu diễn mới của dữ liệu.

1.6. Tại sao Deep Learning cần hàm phi tuyến?

- Nếu mọi layer chỉ thực hiện biến đổi tuyến tính, việc xếp nhiều layer liên tiếp vẫn tương đương với một phép biến đổi tuyến tính-affine duy nhất. Giả sử:

$$h_1 = W_1x + b_1$$

$$h_2 = W_2h_1 + b_2$$

Thay h_1 vào phương trình thứ hai:

$$h_2 = W_2W_1x + W_2b_1 + b_2$$

Biểu thức trên vẫn là một phép biến đổi tuyến tính-affine. Vì vậy, việc chồng nhiều layer tuyến tính không tạo ra khả năng biểu diễn phức tạp tương ứng với một mạng sâu. Hàm activation là yếu tố đưa phi tuyến vào mạng.

Depth + Nonlinearity $\rightarrow$ Expressive Model

1.7. Hàm kích hoạt ReLU

ReLU (Rectified Linear Unit) là một trong những hàm activation phổ biến:

$$\text{ReLU}(z) = \max(0, z)$$

Hay tương đương:

$$\text{ReLU}(z) = 0 \text{ nếu } z < 0; \text{ReLU}(z) = z \text{ nếu } z \geq 0$$

ReLU biến các giá trị âm thành 0 và giữ nguyên các giá trị dương. Đạo hàm được sử dụng trong quá trình học là:

$$\text{ReLU}'(z) = 0 \text{ nếu } z \leq 0; \text{ReLU}'(z) = 1 \text{ nếu } z > 0$$

Ví dụ:

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(0) = 0$$

$$\text{ReLU}(4) = 4$$

Qua ví dụ này có thể thấy ReLU loại bỏ phần âm và giữ nguyên phần dương. Vai trò quan trọng hơn của ReLU là tạo ra tính phi tuyến để mạng có thể học các quan hệ phức tạp.

1.8. Multi-Layer Perceptron và Learned Representation

- Khi nối nhiều layer, ta thu được một Multi-Layer Perceptron (MLP). Một kiến trúc đơn giản có dạng:

$$d \rightarrow 64 \rightarrow 32 \rightarrow C$$

trong đó:

+ d là số chiều đầu vào

+ C là số chiều đầu ra

Layer ẩn đầu tiên có thể được viết:

$$h_1 = \text{ReLU}(W_1x + b_1)$$

Vector h_1 chính là một learned representation. Điểm quan trọng là 64 thành phần của h_1 không được con người đặt tên trực tiếp như age, glucose hay BMI mà được xác định bởi các tham số học được.

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \hat{y}$$

Mỗi layer ẩn thực hiện một learned transformation mới, từ đó tạo ra các representation ngày càng phù hợp hơn với nhiệm vụ.

1.9. Deep Learning: học phân cấp biểu diễn

Một cách diễn đạt quan trọng trong tài liệu là:

Deep Learning = Learning a hierarchy of representations

Với mạng $X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y}$, layer đầu có thể học các mẫu tương đối đơn giản; layer tiếp theo kết hợp các mẫu này thành representation phức tạp hơn; layer cuối ánh xạ representation đã học thành đầu ra của nhiệm vụ.

$$\hat{y} = f_3(f_2(f_1(X)))$$

Biểu thức trên thể hiện function composition và là ý tưởng trung tâm của Deep Learning.

1.10. Độ sâu và độ rộng của mạng

Hai khái niệm dễ nhầm lẫn là width và depth. Width là số neuron trong một layer, còn depth là số phép biến đổi hoặc layer được xếp nối tiếp.

$$8 \rightarrow 100 \rightarrow 1 \quad : \text{rộng nhưng nông}$$

$$8 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 1 \quad : \text{sâu hơn}$$

=> Do đó, “deep” không đơn giản có nghĩa là có rất nhiều neuron. Bản chất của Deep Learning nằm ở nhiều tầng biến đổi và biểu diễn được học nối tiếp nhau. Tài liệu cũng lưu ý rằng nhiều layer hơn không đồng nghĩa chắc chắn với kết quả tốt hơn; kiến trúc phải phù hợp với bài toán.

1.11. Forward Propagation

Quá trình tính toán từ input đến prediction được gọi là forward propagation. Với một mạng gồm hai hidden layer:

$$h_1 = \text{ReLU}(W_1x + b_1)$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2)$$

$$z = W_3h_2 + b_3$$

Luồng tính toán là:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow z$$

Trong ví dụ Deep Learning từ NumPy, luồng đầy đủ được viết:

$$X \rightarrow Z_1 \rightarrow H_1 \rightarrow Z_2 \rightarrow H_2 \rightarrow Z_3 \rightarrow \hat{y}$$

$$Z_1 = XW_1 + b_1$$

$$H_1 = \text{ReLU}(Z_1)$$

$$Z_2 = H_1W_2 + b_2$$

$$H_2 = \text{ReLU}(Z_2)$$

$$Z_3 = H_2W_3 + b_3$$

$$\hat{y} = \sigma(Z_3)$$

1.12. Kích thước ma trận và tensor

Việc hiểu tensor shape là rất quan trọng vì Neural Network về cơ bản là chuỗi phép toán ma trận. Nếu $X \in \mathbb{R}^{N \times 8}$ và $W_1 \in \mathbb{R}^{8 \times 16}$ thì:

$$(N \times 8)(8 \times 16) = N \times 16$$

Do đó $H_1 \in \mathbb{R}^{N \times 16}$. Tiếp theo:

$$(N \times 16)(16 \times 8) = N \times 8$$

nên $H_2 \in \mathbb{R}^{N \times 8}$. Cuối cùng:

$$(N \times 8)(8 \times 1) = N \times 1$$

và $\hat{y} \in \mathbb{R}^{N \times 1}$. Điều này cho thấy mỗi layer biến đổi feature dimension trong khi giữ lại batch dimension N.

Trong ví dụ mini-batch, nếu batch có 32 mẫu thì:

$$32 \times d \rightarrow 32 \times 64 \rightarrow 32 \times 32 \rightarrow 32 \times C$$

Batch dimension là 32 trong suốt quá trình, còn feature dimension thay đổi theo kiến trúc mạng.

1.13. Hàm Sigmoid và đầu ra xác suất

- Đối với bài toán phân loại nhị phân, output thường được đưa qua sigmoid:

$$\sigma(z) = 1 / (1 + e^{-z})$$

- Hàm sigmoid cho giá trị trong khoảng (0, 1), vì vậy output có thể được diễn giải như một probability-like score.

$$0 < \sigma(z) < 1$$

Ví dụ: nếu $\hat{y} = 0.91$ thì mô hình đang gán mức độ tin cậy cao cho class 1. Với threshold 0.5:

$$\hat{y} \geq 0.5 \rightarrow \text{Class 1}$$

$$\hat{y} < 0.5 \rightarrow \text{Class 0}$$

Ví dụ: với biểu diễn $H_2 = [1.2, 0.5, 0, 2.0, 0.4, 0, 1.1, 0.3]^T$ và một vector trọng số đầu ra. Sau khi tính phép biến đổi tuyến tính, thu được xấp xỉ:

$$Z_3 \approx 1.53$$

$$\hat{y} = 1 / (1 + e^{-1.53}) \approx 0.822$$

=> Do đó mô hình dự đoán $P(\text{diabetes}) \approx 0.822$ và với threshold 0.5 thì class dự đoán là 1. Ví dụ này minh họa trực tiếp chuỗi

Hidden Representation $\rightarrow$ Linear Combination $\rightarrow$ Sigmoid $\rightarrow$ Prediction.

1.14. Output của bài toán nhiều lớp và Softmax

- Đối với bài toán phân loại C lớp, mạng có thể tạo ra vector logits:

$$z = [z_1, z_2, \dots, z_C]$$

- Để chuyển sang xác suất, có thể sử dụng Softmax:

$$p_i = e^{z_i} / \sum_j e^{z_j}$$

- Nhân dự đoán thường là lớp có score lớn nhất:

$$\hat{y} = \arg \max_i z_i$$

Khi sử dụng CrossEntropyLoss trong PyTorch, mô hình thường trả trực tiếp logits thay vì tự áp dụng Softmax bên trong model.

1.15. Loss Function

- Forward propagation chỉ tạo ra prediction. Để mô hình học, cần có một đại lượng đo mức độ sai lệch giữa prediction và target. Đại lượng này là loss.

Input → Prediction → Loss

$$\theta^* = \arg \min_{\theta} L(\theta)$$

*Binary Cross-Entropy

- Đối với phân loại nhị phân, Binary Cross-Entropy được định nghĩa:

$$L = -(1/N) \sum_i [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

- Hàm loss phản ánh mức độ sai lệch của prediction. Khi $y = 1$ và $\hat{y} = 0.99$, loss nhỏ. Khi $y = 1$ nhưng $\hat{y} = 0.01$, loss rất lớn. Như vậy loss không chỉ cho biết đúng hay sai mà còn phản ánh mức độ sai lệch và mức độ tự tin của mô hình.

- Trong cài đặt NumPy, giá trị nhỏ $\epsilon = 10^{-8}$ được dùng với clip để tránh các vấn đề số học như $\log(0)$.

1.16. Cross Entropy trong bài toán nhiều lớp

- Với multi-class classification, một loss phổ biến là CrossEntropyLoss. Trong PyTorch có thể sử dụng:

$$L = \text{CrossEntropyLoss}(z, y)$$

Trong đó loss function đo prediction error, còn optimizer quyết định cách thay đổi model parameters để giảm loss. Hai thành phần có vai trò khác nhau nhưng phối hợp trực tiếp trong quá trình học.

1.17. Backpropagation

- Sau khi tính loss, cần xác định mỗi tham số đã góp phần vào sai số như thế nào. Đây là nhiệm vụ của backpropagation. Ví dụ:

$$\partial L / \partial W_3$$

cho biết loss thay đổi như thế nào khi W_3 thay đổi. Tương tự, gradient đối với W_2 và W_1 cho phép xác định ảnh hưởng của các layer phía trước. Cơ sở toán học của backpropagation là chain rule.

$$\partial L / \partial W_1 = (\partial L / \partial H_2)(\partial H_2 / \partial H_1)(\partial H_1 / \partial W_1)$$

Backpropagation vì vậy không phải một phép toán bí ẩn; đó là việc áp dụng có hệ thống chain rule lên một chuỗi các hàm được nối với nhau.

1.18. Backpropagation qua từng layer

- Với mạng ba layer, gradient được truyền ngược từ output về input. Tại layer cuối:

$$dZ_3 = (\hat{y} - y) / N$$

$$dW_3 = H_2^T dZ_3$$

$$db_3 = \Sigma dZ_3$$

Tiếp tục truyền về layer 2:

$$dH_2 = dZ_3 W_3^T$$

$$dZ_2 = dH_2 \odot \text{ReLU}'(Z_2)$$

$$dW_2 = H_1^T dZ_2$$

Cuối cùng:

$$dH_1 = dZ_2 W_2^T$$

$$dZ_1 = dH_1 \odot \text{ReLU}'(Z_1)$$

$$dW_1 = X^T dZ_1$$

- Luồng forward đi theo $X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y} \rightarrow L$, trong khi backward đi ngược từ L về các tham số ở những layer trước.

1.19. Gradient Descent

- Sau khi đã biết gradient, ta cần cập nhật tham số. Với một tham số W :

$$W \leftarrow W - \eta (\partial L / \partial W)$$

trong đó:

+ η là learning rate. Gradient chỉ hướng mà loss tăng, vì vậy việc trừ gradient giúp tham số di chuyển theo hướng làm giảm loss.

Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update

Chu trình này được lặp đi lặp lại nhiều lần trong quá trình huấn luyện.

1.20. Learning Rate

- Learning rate quyết định kích thước của mỗi bước cập nhật. Tài liệu minh họa bằng các giá trị $\eta = 0.001, 0.01$ và 0.1 để quan sát loss curves.

- Nếu learning rate quá nhỏ, quá trình học có thể rất chậm. Nếu learning rate phù hợp, loss có thể giảm ổn định. Nếu learning rate quá lớn, cập nhật có thể quá mạnh và làm quá trình tối ưu không ổn định.

1.21. Chuẩn hóa dữ liệu

- Các đặc trưng có thể nằm trên những thang đo rất khác nhau. Được minh họa việc chuẩn hóa theo công thức:

$$x' = (x - \mu) / \sigma$$

- Trong đó μ và σ được tính từ training data. Training set được chuẩn hóa bằng mean và standard deviation của chính nó, còn test set sử dụng mean và standard deviation của training set. Cách làm này giúp tránh information leakage từ test set vào quá trình huấn luyện và bảo đảm phép biến đổi đầu vào ở test nhất quán với training.

1.22. Training Set, Validation Set và Test Set

- Một mô hình không được đánh giá chỉ dựa trên dữ liệu đã dùng để học. Cần phân biệt Training, Validation và Test. Training data được sử dụng để học tham số; validation data phục vụ theo dõi và lựa chọn thiết kế; test data được giữ lại để đánh giá khả năng tổng quát hóa cuối cùng.

$$\text{Training} \neq \text{Validation} \neq \text{Test}$$

Test set phải là dữ liệu được giữ lại để đưa ra đánh giá cuối cùng về generalization trong thiết kế thí nghiệm.

1.23. Mini-Batch Training

- Thay vì đưa toàn bộ N mẫu vào mạng cùng một lúc, dữ liệu có thể được chia thành các mini-batch:

$$X \rightarrow X_1, X_2, \dots, X_B$$

$$L_b = L(X_b, y_b)$$

Mini-batch giúp sử dụng bộ nhớ hiệu quả hơn, tính toán thuận lợi hơn,

cập nhật tham số thường xuyên hơn và phù hợp với tính toán GPU. Trong PyTorch, Dataset cung cấp dữ liệu mẫu còn DataLoader tổ chức chúng thành mini-batch.

1.24. Training Loop

- Quy trình huấn luyện Neural Network có thể biểu diễn:

Input Batch → Forward → Loss → Backward → Optimizer Step

- Các bước tương ứng trong PyTorch là:

optimizer.zero_grad() → xóa gradient cũ; model(X_batch) → forward propagation; criterion(outputs, y_batch) → tính loss; loss.backward() → backpropagation; optimizer.step() → cập nhật tham số.

Mỗi câu lệnh thực chất tương ứng với một bước toán học trong quy trình tối ưu.

1.25. Training Mode và Evaluation Mode

- PyTorch phân biệt hai trạng thái của mô hình. Trong quá trình huấn luyện sử dụng `model.train()`, còn khi validation hoặc testing sử dụng `model.eval()`. Khi đánh giá, gradient thường không cần thiết nên có thể sử dụng `with torch.no_grad()`. `Model.train()` và `model.eval()` điều khiển operating mode của model, trong khi `torch.no_grad()` tắt việc tính gradient cho phần code bên trong.

1.26. Underfitting và Overfitting

- Underfitting xảy ra khi training performance thấp và validation performance cũng thấp; nguyên nhân có thể là model chưa đủ capacity hoặc optimization chưa phù hợp.

- Overfitting xảy ra khi training performance trở nên rất cao nhưng validation performance dừng cải thiện, training và validation bắt đầu phân kỳ và mô hình có thể ghi nhớ pattern của training data.

Generalization > Training Accuracy Alone

Để theo dõi quá trình học, cần quan sát training loss, validation loss, training accuracy và validation accuracy qua các training curves.

1.27. Đánh giá mô hình

- Mô hình cuối cùng phải được đánh giá trên held-out test data. Đối với

classification, các chỉ số quan trọng gồm Accuracy, Precision, Recall, F1-score và Confusion Matrix.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Confusion Matrix gồm True Positive, True Negative, False Positive và False Negative. Việc quan sát các thành phần này giúp hiểu được loại sai lầm mà mô hình đang mắc phải thay vì chỉ nhìn một con số Accuracy.

1.28. Không nên mặc định Deep Learning luôn tốt hơn Machine Learning

- Neural Network không mặc nhiên tốt hơn mô hình truyền thống. Sự khác biệt có thể được nhìn qua representation, model capacity, nonlinearity, số tham số, interpretability và training cost.

Đặc điểm	Traditional ML	Neural Network
Representation	Prepared	Learned
Model capacity	Thấp hơn	Cao hơn
Nonlinearity	Hạn chế	Có
Số tham số	Ít hơn	Nhiều hơn
Interpretability	Cao hơn	Thấp hơn
Training cost	Thấp hơn	Cao hơn

Bảng 01. So sánh khái quát giữa Traditional ML và Neural Network

Khi so sánh mô hình cần xem xét dataset có đủ lớn hay không, learned representation có thực sự hữu ích hay không, validation và test performance có cải thiện hay không, có bao nhiêu tham số, chi phí tính toán bao nhiêu và mức cải thiện có ý nghĩa thực tế hay không. Vì vậy, mô hình tốt hơn không đồng nghĩa đơn giản với Accuracy cao hơn.

1.29. Deep Learning from Scratch bằng NumPy

- Neural Network được xây dựng từ đầu bằng NumPy, không sử dụng TensorFlow, PyTorch, Keras hoặc scikit-learn cho phần triển khai mạng. Mục đích là làm rõ bản chất của các phép toán bên trong Deep Learning.

Toàn bộ mạng có thể được xây dựng từ matrix multiplication, ReLU, sigmoid, binary cross-entropy, backpropagation và gradient descent.

Python	Toán học
$X @ W_1 + b_1$	$Z_1 = XW_1 + b_1$
$\text{relu}(z_1)$	$H_1 = \text{ReLU}(Z_1)$
$h_1 @ W_2 + b_2$	$Z_2 = H_1W_2 + b_2$
$\text{relu}(z_2)$	$H_2 = \text{ReLU}(Z_2)$
$h_2 @ W_3 + b_3$	$Z_3 = H_2W_3 + b_3$
$\text{sigmoid}(z_3)$	$\hat{y} = \sigma(Z_3)$
$\text{binary_cross_entropy}(\dots)$	L
$\text{backward}(\dots)$	$\nabla \theta L$
$W -= \text{lr} * dW$	$W \leftarrow W - \eta \nabla W L$

Bảng 02. Liên hệ giữa các phép toán trong code NumPy và công thức Deep Learning

1.30. Từ NumPy đến PyTorch

- Sau khi hiểu cách xây dựng mạng thủ công, framework như PyTorch có thể được sử dụng để tự động hóa các phép toán. Một kiến trúc ví dụ có thể được mô tả bằng các layer Linear và ReLU nối tiếp nhau.

Input $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ Output

Về mặt ý tưởng, đây vẫn chính là mạng được xây dựng từ NumPy. Framework chỉ cung cấp cách biểu diễn thuận tiện và hiệu quả hơn. Nó có thể tự động hóa lưu trữ tham số, matrix multiplication, activation, loss, gradient, backpropagation và parameter updates.

Framework $\neq$ Deep Learning itself

Framework = Tools for implementing Deep Learning efficiently

1.31. Lưu mô hình và tái sử dụng

- Sau khi huấn luyện, các tham số đã học có thể được lưu lại. Trong PyTorch, tài liệu sử dụng `state_dict` để lưu trạng thái tham số. Khi sử dụng lại, cần tái tạo đúng architecture rồi load `state_dict` và chuyển model sang evaluation

mode.

```
torch.save(model.state_dict(), "mlp_model.pth")
```

Điều này cho thấy model parameters và model architecture có quan hệ chặt chẽ: state_dict lưu trạng thái tham số, còn kiến trúc phải được tái tạo đúng để các tham số được sử dụng chính xác.

1.32. Learned Representation có thể được phân tích

- Neural Network không chỉ tạo prediction cuối cùng. Hidden representation cũng có thể được trích xuất:

$$h = \text{ReLU}(W_1x + b_1)$$

- Sau đó có thể xem xét $x \in \mathbb{R}^d \rightarrow h \in \mathbb{R}^k$ và đặt ra các câu hỏi: số chiều có thay đổi không, các mẫu thuộc những class khác nhau có tách ra không, pattern nào xuất hiện trong learned space và thông tin ban đầu đã được biến đổi như thế nào.

=> Điều này giúp nhìn Deep Learning không chỉ như một hộp đen nhận input và trả output, mà như một chuỗi các phép biến đổi đã học, trong đó hidden layers là nơi xây dựng các representation trung gian.

1.33. Từ MLP đến các kiến trúc Deep Learning hiện đại

- MLP là nền tảng cơ bản để hiểu Deep Learning, nhưng các hệ thống hiện đại có thể có hàng trăm hoặc hàng nghìn layer, hàng triệu hoặc hàng tỷ tham số, Convolutional layers, Recurrent layers, Attention, Transformers, Residual connections, Normalization, GPU và distributed computation.

Dù kiến trúc hiện đại phức tạp hơn, nguyên lý cơ bản vẫn giữ nguyên:

Compose functions → Calculate loss → Differentiate → Update parameters

Độ phức tạp của hệ thống hiện đại chủ yếu đến từ architecture, scale, data và optimization methods chứ không phải từ việc từ bỏ các nguyên lý cơ bản trên.

1.34. Tensor, hình ảnh và sự mở rộng của Representation Learning

- Representation Learning không giới hạn ở feature vector. Feature vector phù hợp với dữ liệu có cấu trúc, trong khi hình ảnh tự nhiên được biểu diễn dưới dạng tensor:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{X} \in \mathbb{R}^{c \times H \times W}$$

Ví dụ: một ảnh RGB có thể có kích thước $3 \times 224 \times 224$. Nếu flatten ảnh thành một vector rồi đưa vào MLP, một phần cấu trúc không gian có thể bị bỏ qua. Vì vậy CNN được đưa vào để học representation mà vẫn giữ cấu trúc không gian.

$$\text{Vector} \rightarrow \text{MLP}$$

$$\text{Image Tensor} \rightarrow \text{CNN}$$

Ở cấp độ rộng hơn, tiến trình representation có thể được hình dung:

$$\text{Symbols} \rightarrow \text{Features} \rightarrow \text{Vectors} \rightarrow \text{Tensors} \rightarrow \text{Graphs} \rightarrow \text{Embeddings} \rightarrow \text{Multimoda}$$

Feature vectors gắn với traditional ML, tensors gắn với Deep Learning, graphs gắn với GNN và knowledge graphs, embeddings gắn với semantic retrieval và RAG, còn nhiều representation kết hợp dẫn tới multimodal AI.

1.35. Tổng kết lý thuyết Deep Learning

- Toàn bộ lý thuyết có thể cô đọng thành một chuỗi:

$$\mathbf{X} \rightarrow \mathbf{f}_1 \rightarrow \mathbf{H}_1 \rightarrow \mathbf{f}_2 \rightarrow \mathbf{H}_2 \rightarrow \dots \rightarrow \hat{\mathbf{y}}$$

Mỗi $\mathbf{f}_i$ là một learned transformation. Với ví dụ ba layer:

$$\mathbf{H}_1 = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)$$

$$\mathbf{H}_2 = \text{ReLU}(\mathbf{H}_1\mathbf{W}_2 + \mathbf{b}_2)$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{H}_2\mathbf{W}_3 + \mathbf{b}_3)$$

Các tham số $\theta = \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{W}_3, \mathbf{b}_3\}$ được học thông qua:

$$\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \text{Gradient Descent}$$

=> Cách diễn đạt tổng quát:

$$\text{Deep Learning} = \text{Function Composition} + \text{Representation Learning} + \text{Optimization}$$

=> Có thể nhìn toàn bộ quá trình dưới dạng:

$$\text{Data} \rightarrow \text{Representation} \rightarrow \text{Function} \rightarrow \text{Prediction} \rightarrow \text{Loss} \rightarrow \text{Gradient} \rightarrow \text{Update}$$

Đây là nền tảng lý thuyết để hiểu cách Neural Network được xây dựng, huấn

luyện và đánh giá. Người học cần hiểu architecture, tensor shapes, forward propagation, loss, optimizer, backpropagation, training loop và evaluation thay vì chỉ sử dụng các hàm có sẵn.